Adressing Modes

NRP	: 1. 3223600044
	2. 3223600055
	3. 3223600058
Nama	: 1. Devano Auriza Khairindra Heriyanto
	2. Arimbi Kusuma Jati
	3. Ihsanta Zaki Sanjaya
Kelas	: 2 D4 Teknik Komputer B
Matakuliah	: Praktikum Organisasi Mesin dan Bahasa Assembly
Tanggal	: Kamis, 19 September 2024
Dosen	: Dr. Setiawardhana ST, MT

D4 TEKNIK KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2024

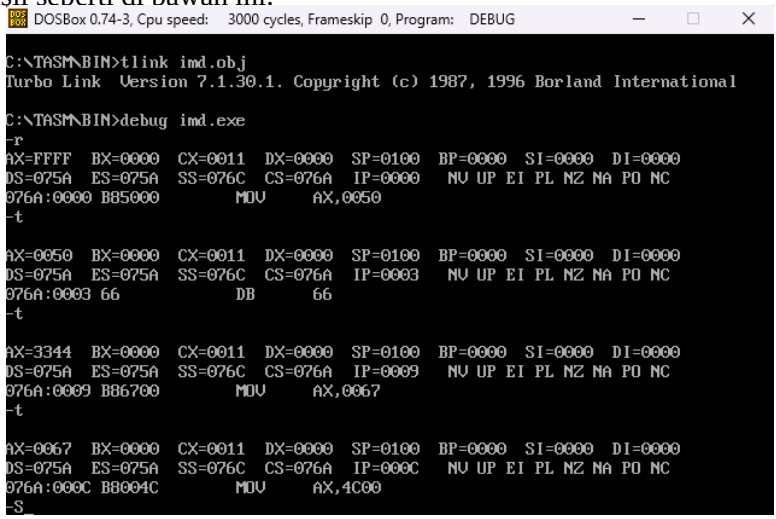
ADDRESSING MODES

1. IMMEDIATE ADDRESSING

Berikut merupakan sintax dari Immediate Addressing yang dapat melakukan pengcopyan data untuk suatu register 8,16 atau 32(80386) bit langsung dari suatu angka.:

```
.MODEL SMALL
.386 ;
.STACK 100h ;
.DATA
A EQU 67h ;
.CODE
START:
MOV AX, 50h ;
MOV EAX, 11223344h ;
MOV AX, A ;
;
MOV AX, 4C00h
INT 21h ;
END START
```

Apabila program dengan sintax diatas di-debug maka akan menampilkan hasil seperti di bawah ini.



Gambar 1 Hasil debug program dengan dosbox

```
AX=0067  BX=0000  CX=0011  DX=0000  SP=0100  BP=0000  SI=0000  DI=0000
DS=075A  ES=075A  SS=076C  CS=076A  IP=000C   NV UP EI PL NZ NA PO NC
076A:000C B8004C        MOV     AX,4C00
-t

AX=4C00  BX=0000  CX=0011  DX=0000  SP=0100  BP=0000  SI=0000  DI=0000
DS=075A  ES=075A  SS=076C  CS=076A  IP=000F   NV UP EI PL NZ NA PO NC
076A:000F CD21          INT     21
```

Gambar 2 Hasil debug program dengan dosbox hingga interupt

Analisa pada program ini adalah dengan memanfaatkan interrupt DOS untuk keluar dari program. Program ini dimulai dengan mendefinisikan nilai simbolik A sebesar 67h, dan melakukan beberapa operasi pemindahan data menggunakan instruksi MOV. Pertama, nilai 50h dimasukkan ke dalam register AX, kemudian nilai 11223344h dimasukkan ke dalam register 32-bit EAX, yang mengakibatkan register AX (bagian bawah dari EAX) diisi dengan nilai 3344h. Setelah itu, nilai simbolik A sebesar 67h dipindahkan ke register AX, menggantikan nilai sebelumnya. Terakhir, nilai 4C00h dimasukkan ke dalam register AX sebagai kode terminasi program, dan interrupt INT 21h digunakan untuk menghentikan eksekusi program secara normal.

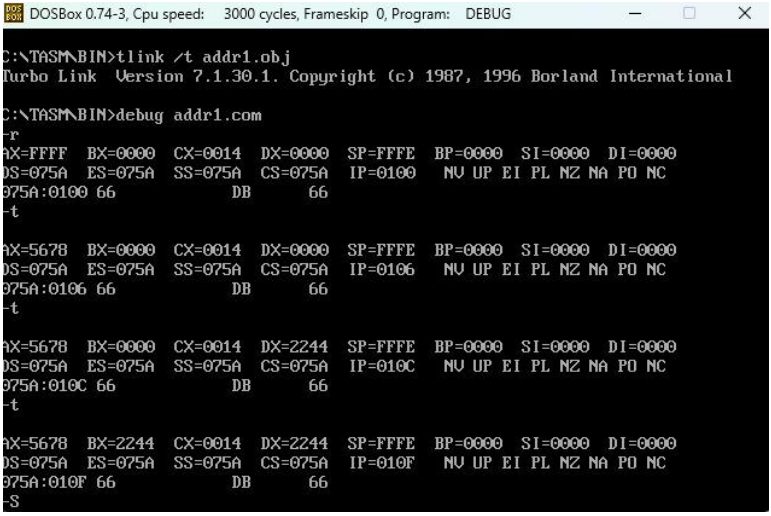
2. REGISTER ADDRESSING

Berikut merupakan sintax dari Register Addressing yang dapat melakukan pengcopyan data.:

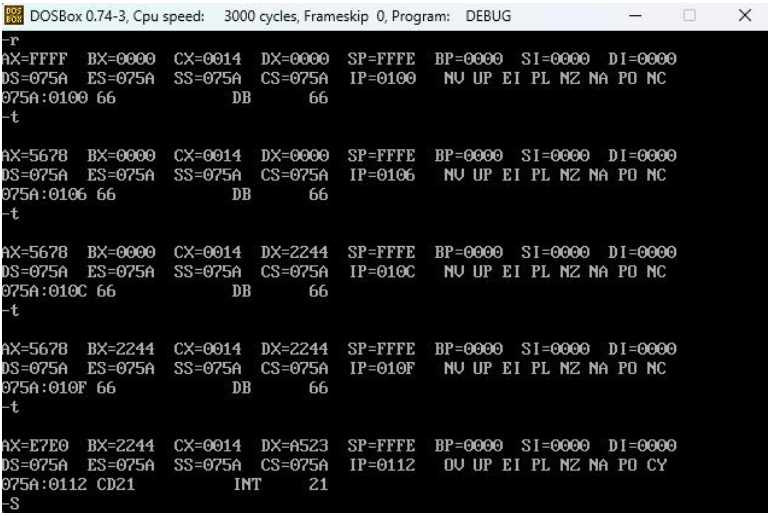
```
;/=========================\;
;   PROGRAM : ADDR1.ASM     ;
;   AUTHOR  : S'to          ;
;   FUNGSI  : PERKALIAN     ;
;             DENGAN 80386 ;
;\=========================/;
          .MODEL SMALL
          .386              ; Untuk prosesor 80386
```

```
            .CODE
            ORG 100h
Proses :
            MOV EAX,12345678h ; Immediate Addressing
            MOV EDX,33112244h ; Immediate Addressing
            MOV EBX,EDX       ; Register  Addressing
            MUL EBX
            INT 21h          ; Register  Addressing
END         Proses
```

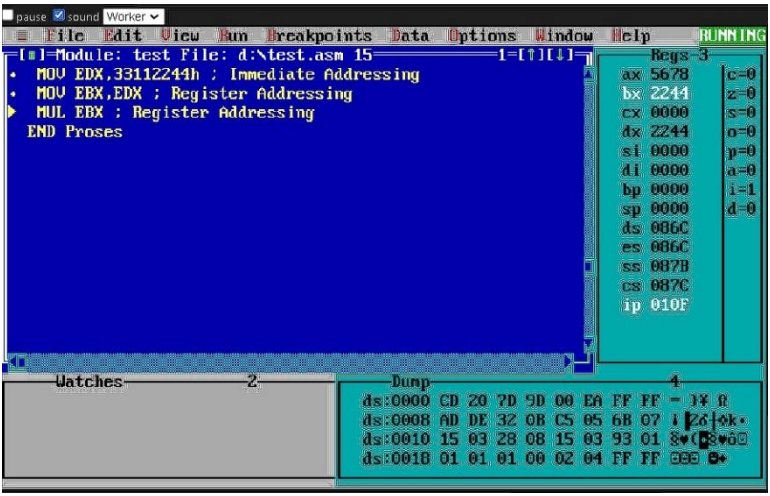
Apabila program dengan sintax diatas di-debug maka akan menampilkan hasil seperti di bawah ini.



Gambar 3 Hasil debug program dengan dosbox



Gambar 4 Hasil debug program dengan dosbox hingga interrupt



Gambar 5 Hasil debug program dengan ekstension MAS/TASM

Analisa pada program percobaan register addressing, dengan melakukan perkalian pada prosesor 80386. Prosesor 80386 perlu dideklarasikan dahulu dengan .386 sehingga program akan melakukan perkalian pada prosesor tersebut. Kemudian, pada bagian proses hal pertama yang perlu dilakukan adalah memberikan nilai pada register dengan cara immediate adressing yang ditandai dengan sintax MOV EAX, 12345678. Setelah sintax dimasukkan maka register AX akan memiliki nilai dari hasil pengcopyan ukuran 1234567 ke dalam

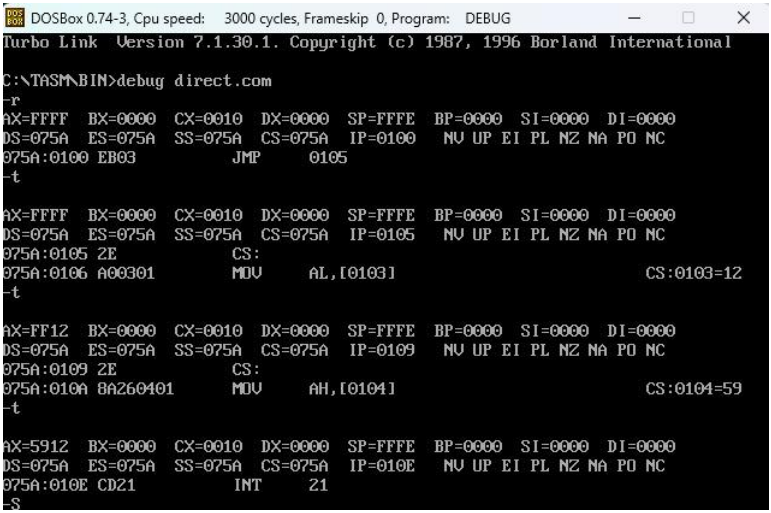
register AX dengan menggunakan MOV. Hal yang sama berlaku pada baris berikutnya yang mengcopy nilai 33112244 ke dalam register DX. Kemudian program menjalankan fungsi utamanya yaitu register adressing yang ditandai dengan sintax MOV EBX, EDX. Sehingga register EBX akan mengcopy nilai dari EDX. Kemudian, ketika pada perintah perkalian atau MUL EBX maka register EBX akan mengalikan dengan register EAX.

3. DIRECT ADDRESSING

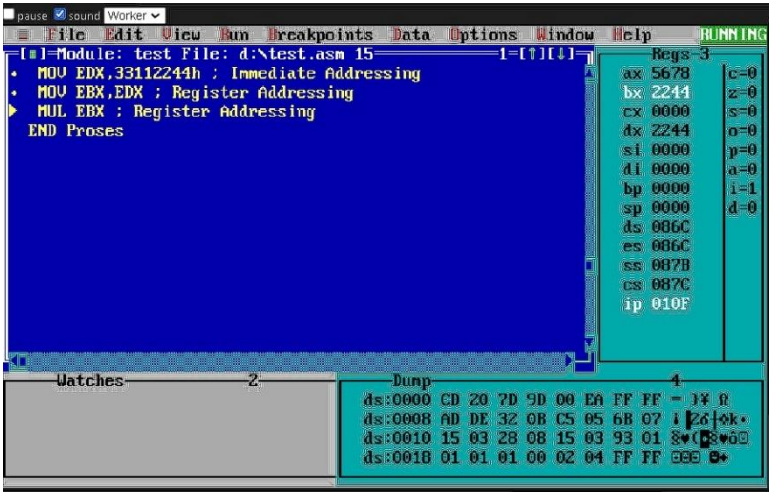
Berikut merupakan sintax dari Direct Addressing yang dapat melakukan pengcopyan data pada suatu register dan simbol.:

```
          .MODEL SMALL
          .386
          .CODE
          ORG 100h
          TData : JMP Proses
                A DB 12h
                B DB 59h
Proses :  MOV AL,A    ; Direct Addressing
                MOV AH,B    ; Direct Addressing
            INT 21h
END       TData
```

Apabila program dengan sintax diatas di-debug maka akan menampilkan hasil seperti di bawah ini.



Gambar 6 Hasil debug program dengan dosbox



Gambar 7 Hasil debug program dengan ekstension MAS/TASM

Analisa pada program percobaan direct addressing, dengan memanfaatkan pengalamatan langsung (direct addressing) untuk memanipulasi data. Program dimulai dengan instruksi JMP Proses, yang melewati deklarasi data variabel A dan B untuk langsung menuju label Proses. Variabel A dan B masing-masing diinisialisasi dengan nilai 12h dan 59h. Pada bagian proses, instruksi MOV AL, A memindahkan nilai dari variabel A (yang beralamatkan langsung) ke

register AL, sedangkan instruksi MOV AH, B memindahkan nilai dari variabel B ke register AH.

4. REGISTER INDIRECT ADDRESSING

Berikut merupakan sintax dari Register Indirect Addressing yang dapat mengakses suatu data yang banyak dengan mengambil alamat efektive dari data tersebut.:

```
;/============================\;
;   PROGRAM : RID.ASM          ;
;   AUTHOR  : S'to             ;
;   FUNGSI  : MENGAKSES DATA   ;
;             MELALUI ALAMAT   ;
;             EFEKTIVE         ;
;\============================/;
        .MODEL SMALL
        .CODE
        ORG 100h
TData : JMP Proses
         Kal DB 'ABCDEF'
Proses:
 Ulang:
         LEA  BX,Kal      ; Ambil Offset Kal
         MOV  CX,2
         MOV  DL,[BX] ; kode ASCII yang ingin dicetak
         MOV  AH,02h    ; Nilai servis ntuk mencetak
karakter
         INT  21h       ; Laksanakan !!
         ADD  BX,2      ; BX:=BX+2
         LOOP Ulang     ; Lompat ke Ulang
         INT 20h
END      TData
```

Apabila program dengan sintax diatas di-debug maka akan menampilkan hasil seperti di bawah ini.

```
DOSBox 0.74-3, Cpu speed:   3000 cycles, Frameskip  0, Program:   DEBUG

C:\TASM\BIN>debug rid.com
-r
AX=FFFF  BX=0000  CX=001C  DX=0000  SP=FFFE  BP=0000  SI=0000  DI=0000
DS=075A  ES=075A  SS=075A  CS=075A  IP=0100   NV UP EI PL NZ NA PO NC
075A:0100 EB07          JMP     0109
-t

AX=FFFF  BX=0000  CX=001C  DX=0000  SP=FFFE  BP=0000  SI=0000  DI=0000
DS=075A  ES=075A  SS=075A  CS=075A  IP=0109   NV UP EI PL NZ NA PO NC
075A:0109 BB0301        MOV     BX,0103
-t

AX=FFFF  BX=0103  CX=001C  DX=0000  SP=FFFE  BP=0000  SI=0000  DI=0000
DS=075A  ES=075A  SS=075A  CS=075A  IP=010C   NV UP EI PL NZ NA PO NC
075A:010C B90200        MOV     CX,0002
-t

AX=FFFF  BX=0103  CX=0002  DX=0000  SP=FFFE  BP=0000  SI=0000  DI=0000
DS=075A  ES=075A  SS=075A  CS=075A  IP=010F   NV UP EI PL NZ NA PO NC
075A:010F 8A17          MOV     DL,[BX]                          DS:0103=41
```

Gambar 8 Hasil debug program dengan dosbox

```
DOSBox 0.74-3, Cpu speed:   3000 cycles, Frameskip  0, Program:   DEBUG

AX=FFFF  BX=0000  CX=001C  DX=0000  SP=FFFE  BP=0000  SI=0000  DI=0000
DS=075A  ES=075A  SS=075A  CS=075A  IP=0109   NV UP EI PL NZ NA PO NC
075A:0109 BB0301        MOV     BX,0103
-t

AX=FFFF  BX=0103  CX=001C  DX=0000  SP=FFFE  BP=0000  SI=0000  DI=0000
DS=075A  ES=075A  SS=075A  CS=075A  IP=010C   NV UP EI PL NZ NA PO NC
075A:010C B90200        MOV     CX,0002
-t

AX=FFFF  BX=0103  CX=0002  DX=0000  SP=FFFE  BP=0000  SI=0000  DI=0000
DS=075A  ES=075A  SS=075A  CS=075A  IP=010F   NV UP EI PL NZ NA PO NC
075A:010F 8A17          MOV     DL,[BX]                          DS:0103=41
-t

AX=FFFF  BX=0103  CX=0002  DX=0041  SP=FFFE  BP=0000  SI=0000  DI=0000
DS=075A  ES=075A  SS=075A  CS=075A  IP=0111   NV UP EI PL NZ NA PO NC
075A:0111 B402          MOV     AH,02
-t

AX=02FF  BX=0103  CX=0002  DX=0041  SP=FFFE  BP=0000  SI=0000  DI=0000
DS=075A  ES=075A  SS=075A  CS=075A  IP=0113   NV UP EI PL NZ NA PO NC
075A:0113 CD21          INT     21
-S_
```

Gambar 9 Hasil debug program dengan dosbox hingga interrupt

Gambar 10 Hasil debug program dengan ekstension MAS/TASM

Analisa pada program percobaan register indirect addressing, dengan mengakses data dengan menggunakan alamat efektif dari data. Karena percobaan ini tidak memasukkan directive .386 maka program tidak dapat berjalan di prosessor 80386. Oleh karenanya digunakan register 32 bit seperti AH, BX, CX, dan DB. Proses pertama yang dilakukan adalah mendeklarasikan data yang akan ditunjuk yang pada percobaan ini adalah Kal DB 'ABCDEF'. Setelahnya mulai dibangun proses. Proses yang pertama adalah Register Indirect Addressing dengan sintax LEA BX, KAL. LEA di sini merupakan Load Effective Address sehingga memberi perintah pada BX untuk mengisi register dengan nilai alamat offset pada variabel 'kal' yang tersimpan di memory. Karena tidak ada perintah khusus maka BX akan menunjuk alamat offset secara default yaitu 'A' pada memory 'kal'. Kemudian register CX diberi nilai 2 sebagai nilai counter pada saat loop. Untuk mencetak nilai yang ditunjuk oleh BX maka pertama perlu untuk menuliskan kode yang akan menampilkan nilai BX. MOV DL, [BX] akan menyimpan kode yang dicetak pada DL. Kemudian perintah mencetak data dilakukan dengan memasukkan nilai servis berikut,

MOV AH, 02h;

INT 21h;.

Sampai pada titik ini maka data dari alamat yang ditunjuk sudah tercetak. Kemudian percobaan untuk menggeser alamat yang ditunjuk menggunakan format penjumlahan atau ADD. Sehingga nilai BX akan bertambah 2 dan menggeser sebanyak dua kali alamat offset. Untuk kembali menampilkan data setelah alamat tergeser digunakan perintah loop yang akan mengulang perintah sebanyak 2 kali sesuai dengan nilai counter yang telah diberikan. Bila setelah 2 kali loop maka program akan kembali pada interupt dan kemudian kembali ke DOS.

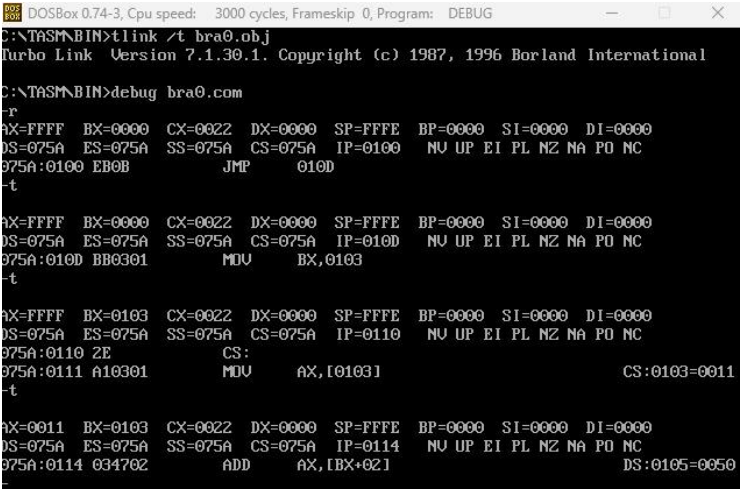
5. BASE RELATIVE (BRA0) ADDRESSING

Berikut merupakan sintax dari Register Indirect Addressing yang dapat mengakses suatu tabel dengan mengambil alamat efektivenya.:

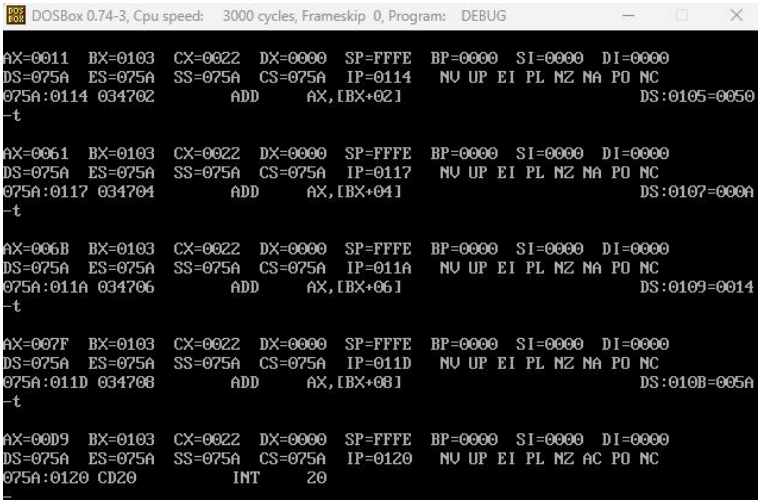
```
;/==============================\;
;   PROGRAM : BRA0.ASM           ;
;   AUTHOR  : S'to               ;
;   FUNGSI  : MENGAKSES DATA     ;
;             DENGAN BASE        ;
;             RELATIVE ADDRESSING;
;                                ;
;\==============================/;
     .MODEL SMALL
     .CODE
      ORG 100h
TData : JMP Proses
        Tabel DW 11h,50h,0Ah,14h,5Ah
Proses:
        LEA  BX,Tabel
        MOV  AX,Tabel
        ADD  AX,[BX]+2
        ADD  AX,[BX]+4
```

```
        ADD  AX,[BX+6]
        ADD  AX,[BX+8]
        INT 20h
END     TData
```

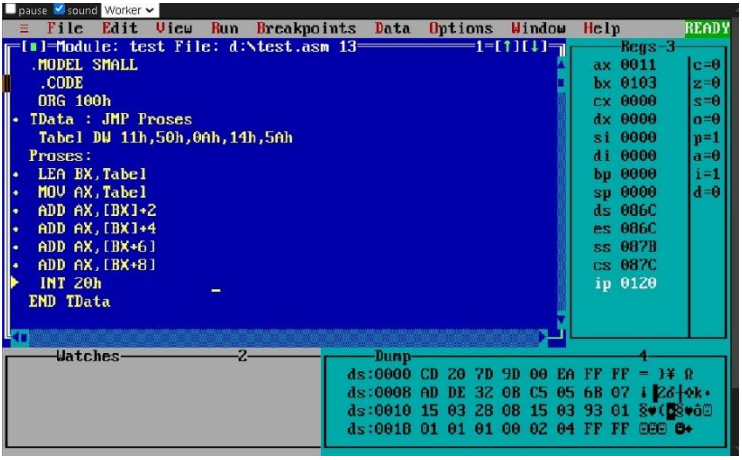
Apabila program dengan sintax diatas di-debug maka akan menampilkan hasil seperti di bawah ini.



Gambar 11 Hasil debug program dengan dosbox



Gambar 12 Hasil debug program dengan dosbox hingga interrupt



Gambar 13 Hasil debug program dengan ekstension MAS/TASM

Analisa pada program percobaan base relative addressing, cara kerjanya mirip dengan Register Indirect Addressing yaitu mengakses dengan mengambil nilai dari alamat efektif. Perbedaan dari Addressing ini adalah digunakan untuk mengakses suatu tabel berbeda dengan RID yang mengakses data yang banyak. Untuk memulai program maka tabel perlu dideklarasikan dengan sintax

Tabel DW 11h, 50h, 0Ah, 14h, 5Ah. Proses dimulai dengan

LEA BX, Tabel

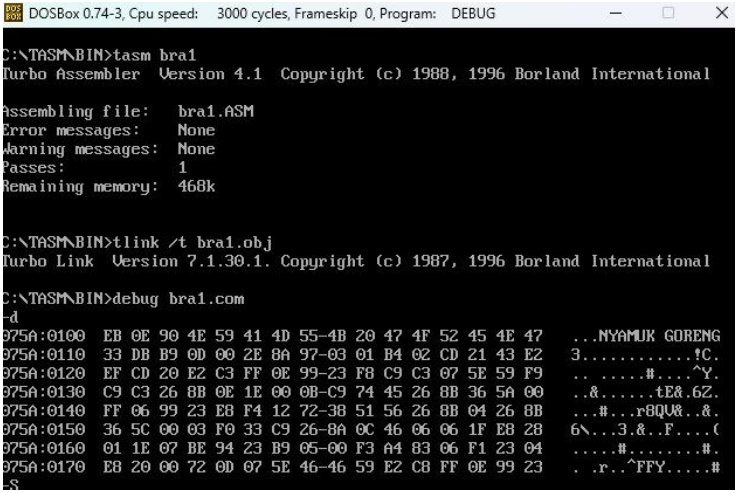
Perintah ini akan membuat BX mengambil alamat offset dari tabel secara default atau berurutan. Ketika BX sudah menunjuk alamat dari tabel maka dengan perintah MOV AX, Tabel register AX akan berisi nilai pada alamat Tabel yang telah ditunjuk. Perlu dicatat bahwa pada adressing yang menunjuk alamat perlu digunakan kurung siku '[ ]' sebagai penanda bahwa yang diambil bukan nilai BX namun nilai pada alamat yang ditunjuk oleh BX. Pada sintax ADD AX,[BX] +2 dan ADD AX, [BX+6] apabila program dijalankan maka tabel yang ditunjuk tetap dan tidak berubah sehingga pengaturan ADD digunakan karena nilai tabel bernilai 2 word. Untuk catatan bahwa penulisan '+' baik di dalam maupun di luar kurung siku tidak menunjukkan perbedaan

6. BASE RELATIVE (BRA1) ADDRESSING

Berikut merupakan sintax dari Register Indirect Addressing yang dapat mengakses suatu tabel dengan mengambil alamat efektivenya.:

```
;/==============================\;
;   PROGRAM : BRA1.ASM           ;
;   AUTHOR  : S'to               ;
;   FUNGSI  : MENCETAK KALIMAT   ;
;             DENGAN BASE        ;
;             RELATIVE ADDRESSING;
;                                ;
;\==============================/;
        .MODEL SMALL
        .CODE
        ORG 100h
TData : JMP Proses
        Kalimat DB 'NYAMUK GORENG'   ; 13 karakter
Proses:
Ulang :
        XOR  BX,BX           ; BX=0 Untuk penunjuk
Offset
        MOV  CX,13           ; Counter LOOP
        MOV  DL,Kalimat[BX]  ; Ambil karakter yang ke BX
        MOV  AH,02           ; Servis untuk cetak
karakter
        INT  21h             ; Cetak Karakter
        INC  BX              ; BX:=BX+1
        LOOP Ulang           ; Lompat ke Ulang sampai
CX=0
        INT 20h              ; Selesai, kembali ke DOS
!!
END     TData
```

Apabila program dengan sintax diatas di-debug maka akan menampilkan hasil seperti di bawah ini.



Gambar 14 Hasil debug program dengan dosbox

Analisa pada program percobaan base relative addressing (bra1), dengan mencetak karakter menggunakan Base Relative Addressing. Kalimat yang akan dicetak dideklarasikan dahulu dengan,

Kalimat DB 'NYAMUK GORENG'

Kemudian register BX perlu di XOR kan dengan XOR sehingga bernilai 0 sebagai step reset BX. Selanjutnya, register CX akan diberi nilai 13 sesuai dengan jumlah karakter yang akan ditampilkan. MOV DL, Kalimat [BX] memerintahkan untuk DL diisi dengan nilai yang diambil dari variabel 'kalimat' kemudian untuk menampilkan karakter digunakan servis

MOV AH, 02

INT 21h. Untuk menampilkan nilai berikutnya maka BX digeser satu kali dengan perintah INC BX yang akan menambah BX dengan 1. Untuk menampilkan semua karakter secara lengkap maka digunakan perintah LOOP yang akan terus mengulangi langkah penunjukkan hingga 13 kali sesuai dengan karakter pada CX atau hingga CX=0 lalu kemudian akan kembali ke DOS karena INT 21h.

7. DIRECT INDEXED ADDRESSING

Berikut merupakan sintax dari Register Indirect Addressing yang dapat mengambil alamat efektive dari suatu data dan mengakses data dengan menggunakan register DI atau SI.:

```
;   PROGRAM : VRBIOS.ASM     ;
;   AUTHOR  : S'to           ;
;   FUNGSI  : MELIHAT VERSI  ;
;             BIOS KOMPUTER  ;
;                            ;
;\==========================/;
         .MODEL SMALL
         .CODE
         ORG 100h
Proses :
  Ulang:
         MOV AX,0F000h       ; Masukkan nilai F000 pada
AX
         MOV ES,AX           ; Copykan nilai AX ke ES
         MOV BX,0FFF5h       ; Penunjuk Offset
         XOR SI,SI           ; Jadikan SI=0
         MOV CX,8            ; Counter untuk LOOP
         MOV  DL,ES:[BX][SI] ; Ambil isi alamat ES:BX+SI
         MOV  AH,02h         ; Nilai servis mencetak
karakter
         INT  21h            ; Cetak !!
         INC  SI             ; SI:=SI+1
         LOOP Ulang          ; Lompat ke Ulang sampai
CX=0
         INT  20h            ; Selesai ! kembali ke DOS
END      Proses
```

Apabila program dengan sintax diatas di-debug maka akan menampilkan hasil seperti di bawah ini.

```
DOSBox 0.74-3, Cpu speed:   3000 cycles, Frameskip  0, Program:   DEBUG

C:\TASM\BIN>tlink /t vrbios.obj
Turbo Link  Version 7.1.30.1. Copyright (c) 1987, 1996 Borland International

C:\TASM\BIN>debug vrbios.com
-r
AX=FFFF  BX=0000  CX=0019  DX=0000  SP=FFFE  BP=0000  SI=0000  DI=0000
DS=075A  ES=075A  SS=075A  CS=075A  IP=0100   NV UP EI PL NZ NA PO NC
075A:0100 B800F0        MOV     AX,F000
-t

AX=F000  BX=0000  CX=0019  DX=0000  SP=FFFE  BP=0000  SI=0000  DI=0000
DS=075A  ES=075A  SS=075A  CS=075A  IP=0103   NV UP EI PL NZ NA PO NC
075A:0103 8EC0          MOV     ES,AX
-t

AX=F000  BX=0000  CX=0019  DX=0000  SP=FFFE  BP=0000  SI=0000  DI=0000
DS=075A  ES=F000  SS=075A  CS=075A  IP=0105   NV UP EI PL NZ NA PO NC
075A:0105 BBF5FF        MOV     BX,FFF5
-t

AX=F000  BX=FFF5  CX=0019  DX=0000  SP=FFFE  BP=0000  SI=0000  DI=0000
DS=075A  ES=F000  SS=075A  CS=075A  IP=0108   NV UP EI PL NZ NA PO NC
075A:0108 33F6          XOR     SI,SI
-S
```

Gambar 15 Hasil debug program dengan dosbox

```
DOSBox 0.74-3, Cpu speed:   3000 cycles, Frameskip  0, Program:   DEBUG
AX=F000  BX=FFF5  CX=0019  DX=0000  SP=FFFE  BP=0000  SI=0000  DI=0000
DS=075A  ES=F000  SS=075A  CS=075A  IP=0108   NV UP EI PL NZ NA PO NC
075A:0108 33F6          XOR     SI,SI
-t

AX=F000  BX=FFF5  CX=0019  DX=0000  SP=FFFE  BP=0000  SI=0000  DI=0000
DS=075A  ES=F000  SS=075A  CS=075A  IP=010A   NV UP EI PL ZR NA PE NC
075A:010A B90800        MOV     CX,0008
-t

AX=F000  BX=FFF5  CX=0008  DX=0000  SP=FFFE  BP=0000  SI=0000  DI=0000
DS=075A  ES=F000  SS=075A  CS=075A  IP=010D   NV UP EI PL ZR NA PE NC
075A:010D 26            ES:
075A:010E 8A10          MOV     DL,[BX+SI]                          ES:FFF5=30
-t

AX=F000  BX=FFF5  CX=0008  DX=0030  SP=FFFE  BP=0000  SI=0000  DI=0000
DS=075A  ES=F000  SS=075A  CS=075A  IP=0110   NV UP EI PL ZR NA PE NC
075A:0110 B402          MOV     AH,02
-t

AX=0200  BX=FFF5  CX=0008  DX=0030  SP=FFFE  BP=0000  SI=0000  DI=0000
DS=075A  ES=F000  SS=075A  CS=075A  IP=0112   NV UP EI PL ZR NA PE NC
075A:0112 CD21          INT     21
-S
```

Gambar 16 Hasil debug program dengan dosbox hingga interrupt

Analisa pada program ini dengan memulai eksekusi di alamat offset 100h dan pada bagian prosesnya, nilai F000h dimasukkan ke dalam register AX kemudian disalin ke register segment ES untuk menunjukkan segment BIOS. Offset BX diatur ke 0FFF5h, menunjuk ke lokasi dalam memori yang kemungkinan menyimpan informasi versi BIOS. Register SI diinisialisasi menjadi 0, dan register CX diatur dengan nilai 8 sebagai counter untuk loop, menunjukkan bahwa program akan menampilkan 8 karakter. Pada setiap iterasi loop, karakter dari alamat ES:BX+SI diambil dan disimpan di register DL, lalu ditampilkan di layar menggunakan layanan interrupt INT 21h dengan servis AH=02h untuk mencetak karakter. Setelah setiap iterasi, nilai SI dinaikkan untuk mengakses karakter berikutnya, dan loop akan berjalan hingga CX mencapai nol. Program berakhir dengan interrupt INT 20h yang mengembalikan kontrol ke DOS.

8. BASE RELATIVE (BRA0) ADDRESSING

Berikut merupakan sintax dari Register Indirect Addressing yang dapat mengakses suatu record atau suatu array 2 dimensi:

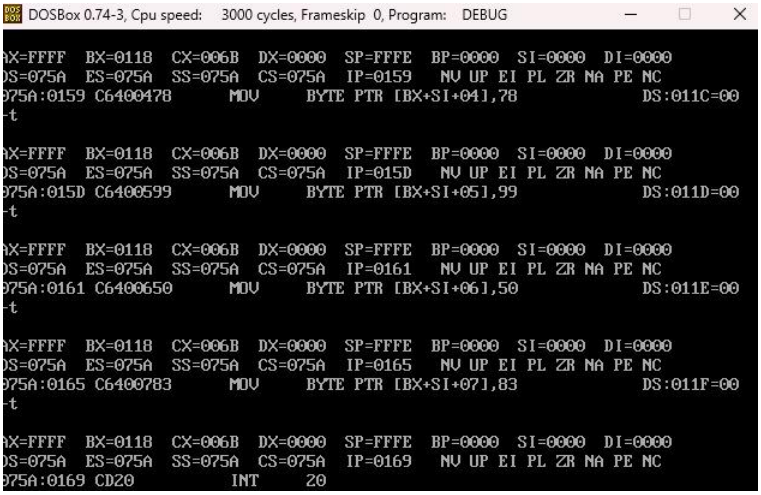
```
;/==============================\;
;   PROGRAM : BIA.ASM             ;
;   AUTHOR  : S'to                ;
;   FUNGSI  : MENGAKSES ARRAY     ;
;             DENGAN BASE         ;
;             INDEXED ADDRESSING  ;
;\==============================/;
         .MODEL SMALL
         .CODE
         ORG 100h
TData : JMP  Proses
        Mahasiswa  STRUC
                  Nim      DW  0        ; 2 byte
                  Tinggi   DB  0        ; 1 byte
                  Nilai    DB  0,0,0,0  ; 4 byte
```

```
        Mahasiswa  ENDS
        Absen      Mahasiswa 10 DUP (<>)
Proses:
        LEA  BX,Absen       ; BX Menunjuk Offset Absen
        ADD  BX,21          ; BX Menunjuk pada Record ke
4
        XOR  SI,SI          ; SI=0
        MOV  [BX][SI].Nim     ,0099h ; NIM, record ke 4
        MOV  [BX][SI].Tinggi ,10h    ; Tinggi, record ke
4
        MOV  [BX][SI+1].Nilai,78h    ; Nilai pertama
        MOV  [BX][SI+2].Nilai,99h    ; Nilai kedua
        MOV  [BX][SI+3].Nilai,50h    ; Nilai keempat
        MOV  [BX][SI+4].Nilai,83h    ; Nilai kelima
        INT  20h                     ; Selesai !!
END     TData
```

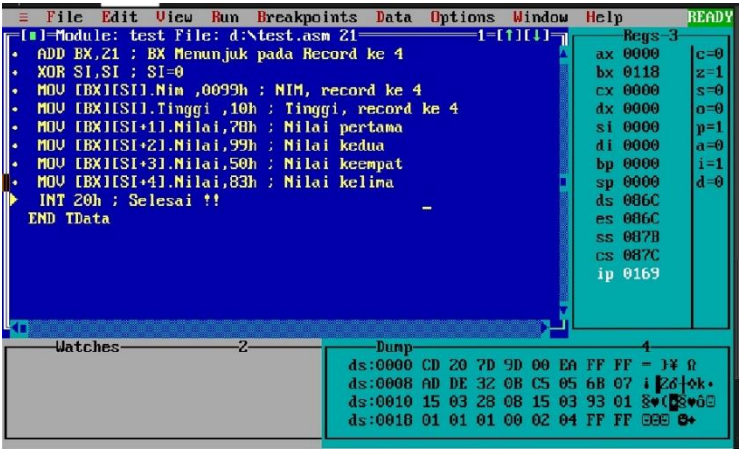
Apabila program dengan sintax diatas di-debug maka akan menampilkan hasil seperti di bawah ini.

```
DOSBox 0.74-3, Cpu speed:   3000 cycles, Frameskip  0, Program:   DEBUG

C:\TASM\BIN>tlink /t bia.obj
Turbo Link  Version 7.1.30.1. Copyright (c) 1987, 1996 Borland International

C:\TASM\BIN>debug bia.com
-r
AX=FFFF  BX=0000  CX=006B  DX=0000  SP=FFFE  BP=0000  SI=0000  DI=0000
DS=075A  ES=075A  SS=075A  CS=075A  IP=0100   NV UP EI PL NZ NA PO NC
075A:0100 EB47          JMP     0149
-t

AX=FFFF  BX=0000  CX=006B  DX=0000  SP=FFFE  BP=0000  SI=0000  DI=0000
DS=075A  ES=075A  SS=075A  CS=075A  IP=0149   NV UP EI PL NZ NA PO NC
075A:0149 BB0301        MOV     BX,0103
-t

AX=FFFF  BX=0103  CX=006B  DX=0000  SP=FFFE  BP=0000  SI=0000  DI=0000
DS=075A  ES=075A  SS=075A  CS=075A  IP=014C   NV UP EI PL NZ NA PO NC
075A:014C 83C315        ADD     BX,+15
-t

AX=FFFF  BX=0118  CX=006B  DX=0000  SP=FFFE  BP=0000  SI=0000  DI=0000
DS=075A  ES=075A  SS=075A  CS=075A  IP=014F   NV UP EI PL NZ NA PE NC
075A:014F 33F6          XOR     SI,SI
-S
```

Gambar 17 Hasil debug program dengan dosbox



Gambar 18 Hasil debug program dengan dosbox hingga interrupt



Gambar 19 Hasil debug program dengan ekstension MAS/TASM

Analisa pada program percobaan base indexed addressing, mendefinisikan struktur Mahasiswa, yang terdiri dari field Nim (2 byte), Tinggi (1 byte), dan Nilai (4 byte), dan menduplikasinya sebanyak 10 kali dalam array Absen. Di bagian proses, program menggunakan instruksi LEA BX, Absen untuk memuat alamat dasar

dari array Absen ke register BX, kemudian menambahkan offset 21 agar menunjuk ke record mahasiswa ke-4. Register SI diinisialisasi menjadi 0 dan digunakan sebagai offset tambahan untuk mengakses field di dalam record tersebut. Program memodifikasi Nim mahasiswa ke-4 dengan nilai 0099h, Tinggi dengan nilai 10h, dan masing-masing dari empat nilai pada field Nilai dengan nilai 78h, 99h, 50h, dan 83h. Setelah semua data diakses dan dimodifikasi menggunakan metode base indexed addressing, program berakhir dengan interrupt INT 20h untuk kembali ke DOS.

KESIMPULAN

Pada praktikum ini, kami berhasil mengimplementasikan berbagai teknik addressing modes dalam bahasa Assembly, mulai dari immediate addressing, register addressing, direct addressing, hingga base indexed addressing. Melalui beberapa program percobaan, kami mempelajari cara mengakses dan memanipulasi data secara langsung menggunakan register dan alamat memori. Dalam immediate addressing, nilai segera di-copy ke register tanpa mengambil dari memori, sedangkan pada register addressing, data dipindahkan antar register. Direct addressing memungkinkan pengaksesan langsung ke variabel memori, dan register indirect addressing menggunakan register untuk menunjuk ke alamat memori secara tidak langsung. Base relative addressing melibatkan penggunaan register sebagai penunjuk dasar dengan offset tertentu, serta base indexed addressing menggabungkan base dan offset untuk mengakses data yang lebih kompleks seperti array dan struktur. Praktikum ini juga menunjukkan pentingnya penggunaan interrupt DOS untuk mengakhiri program secara tepat. Implementasi ini memberikan pemahaman yang mendalam tentang cara komputer mengakses memori dan data melalui berbagai teknik pengalamatan.